



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 6

**Алгоритми за търсене на елемент в
структура от данни. Алгоритми за
последователно и двоично търсене.
Хеширане.**

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ



Алгоритми за търсене

Алгоритмите за търсене се явяват най-често срещани съставни части на системно и потребителско програмно осигуряване.

Търсенето е процес на преглеждане на неподредена, или подредена по определен ключ или ключове съвкупност от данни. Търсенето, както и сортирането на данни, може да бъде **вътрешно**, когато данните са разположени в оперативната памет, и **външно**, когато данните или част от тях се намира на някакъв външен носител.

Ограничаваме се с разглеждането на алгоритми за вътрешно търсене.

Данните в примерите представляват цели числа, организирани в масив, или таблица. Таблицата може да бъде подредена или неподредена.

Максималната сложност на алгоритмите за търсене е $O(n)$. Това е очевидно, т. к. максималният брой необходими сравнения е n .



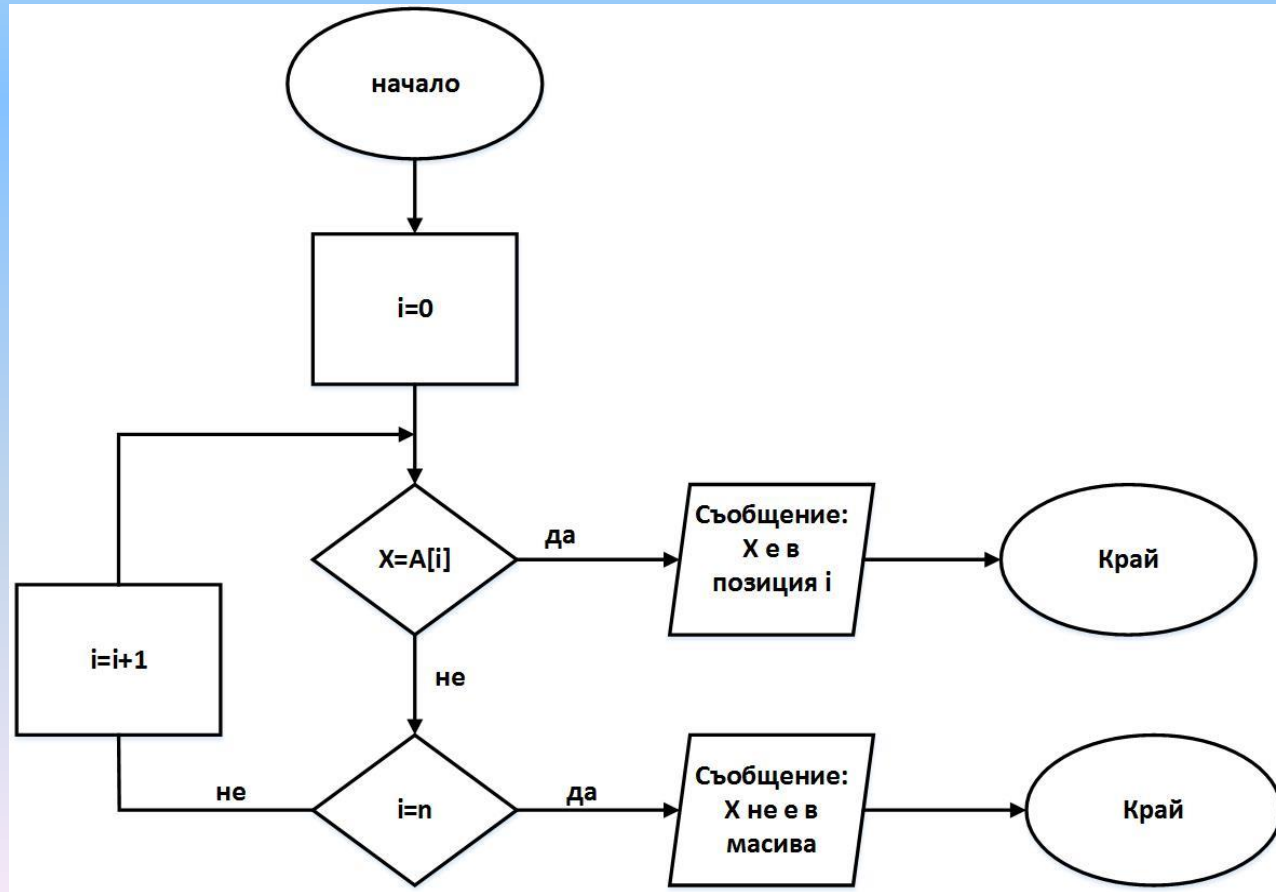
Алгоритми за търсене. Неподреден масив.

Принципни алгоритми за **търсене на елемент** със **стойност x**
в **неподреден масив**:

1 метод: Блок-схема на алгоритъм с **последователно обхождане**
на всички елементи:

В най-лошия случай
имаме **$2n$ сравнения**.

$$T_{\max} = 2n$$





Алгоритми за търсене. Неподреден масив.

Принципни алгоритми за **търсене на елемент** със **стойност x**
в **неподреден масив**:

2 метод: Блок-схема на алгоритъм с **последователно обхождане**
на **всички елементи**:

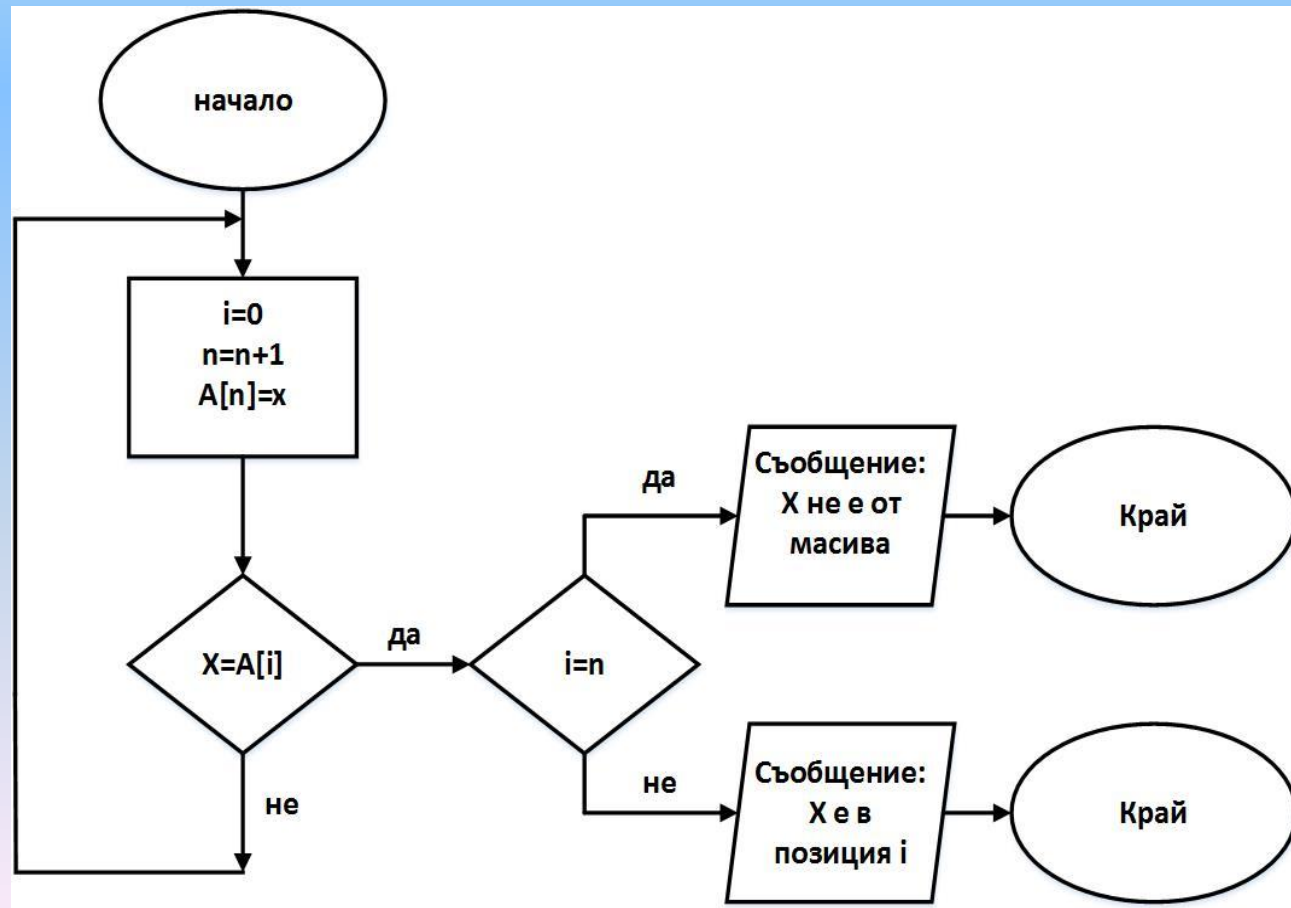
Добавя се **нов елемент** със **стойност x** ;

$n=n+1$; $A[n]=x$;

$i=0,1,2,\dots,n,n+1$;

Сравнението $i=n$
отпада и

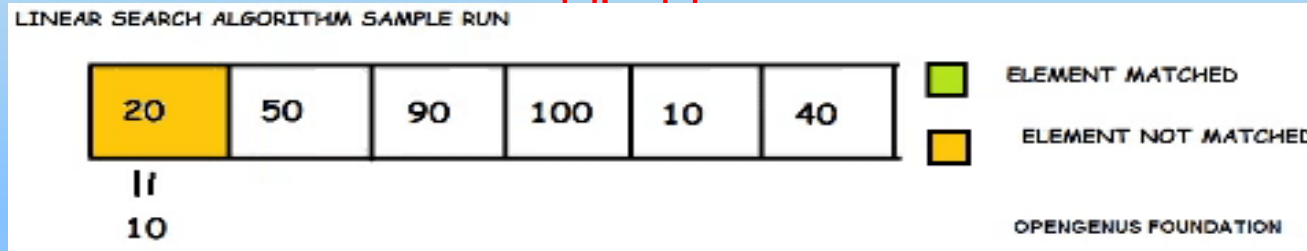
$T_{\max} = n+1$





Алгоритми за търсене. Неподреден масив.

Конкретна реализация на **търсене на елемент** със **стойност x** в **неподреден масив**:



Линейно (последователно) търсене (Linear Search, Sequential Search):

```
int seq_search(int a[], int x)
// a[] - неподредена последователност от цели числа
// x - търсената стойност
{
    int i=0; //работна променлива-индекс
    while ((a[i]!=x) && (i<num)) //num-броят на елементите
        i++;
    if (a[i]==x)
        return 1; //елементът е намерен
    else return 0; //елементът не е намерен
}
```



Алгоритми за търсене. Неподреден масив.

Линейно (последователно) търсене (Linear Search, Sequential Search):

Ако търсенето е **успешно**, **средният брой** на извършваните сравнения е **$n/2$** , ако предположим, че търсеният елемент равновероятно може да се намира на произволно място в масива (**n** - броят на елементите в масива). В най-лошия случай са необходими **n** операции за сравнение, ако търсенето е неуспешно.

Максималната сложност на алгоритъма е **$O(n)$** в случая, ако търсеният елемент е **последен или не е включен в таблицата**. **Минималната сложност** е **$O(1)$** , ако търсеният елемент е **първи** в таблицата.



Алгоритми за търсене. Сортиран масив.

Има **3 метода** за търсене на елемент **x** в **подреден масив**:

2 последователни метода и

1 метод на двоично търсене.

1. Търси се последователно, докато се стигне до **елемент**, който е **по-голям от x** или докато **i** стане **> n**.

Следователно в **най-лошия случай** имаме **2n** сравнения.

2. **Записва** се теоритично **безкрайна стойност** в **n+1 позиция** или **стойност по-голяма** от стойността на елемент **A[n]**. Сравнението **i=n** отпада и следователно имаме

максимум n+1 сравнения.



Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

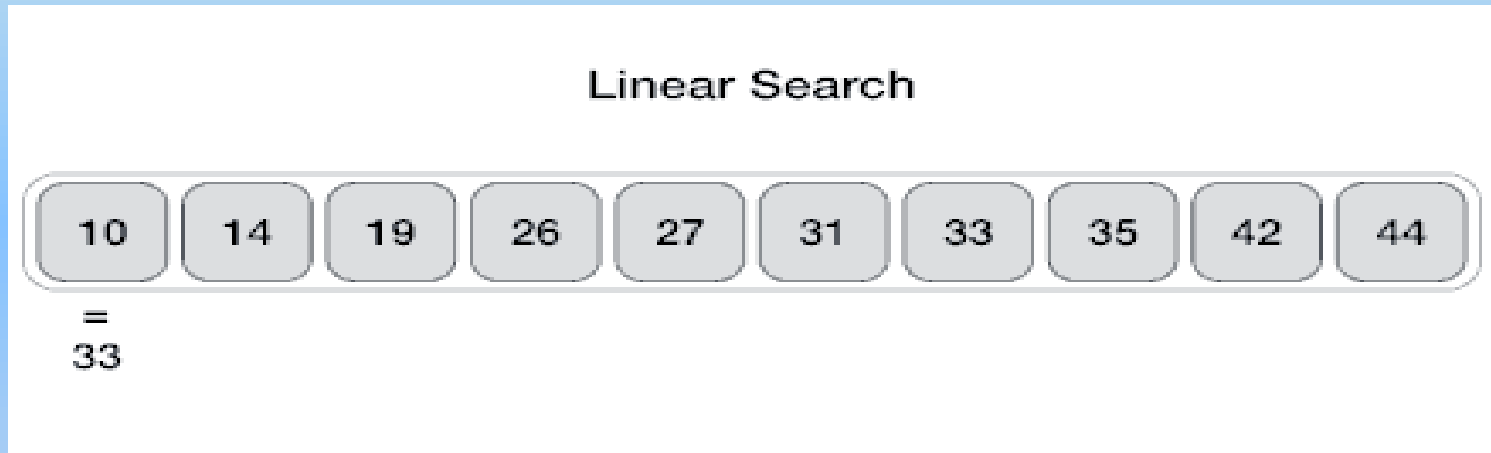
Пр1. *Линейно (последователно) търсене в подредена таблица (Ordered Table Searching)* - Ако таблицата съдържа подредени данни, не е необходимо да се преглежда цялата последователност. Търсенето може да бъде прекратено предварително.

```
int seq_order_search(int a[], int x)           //a[] - подредена
                                              //последователност от цели числа
                                              //x - търсената стойност
{ int i=0;                                    //работна променлива-индекс
  while ((a[i]<x)&&(i<n)) i++;                //n - броят на елементите
  if (a[i]==x)                                //елементът е намерен
    return 1;
  else return 0;                             //елементът не е намерен
}
```



Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:



Търсенето се прекратява при **достигането на търсения елемент** или **първия по-голям от него**, което означава, че елементът не е включен в таблицата.

Броят на необходимите **сравнения** в случай на **неуспешно търсене** намалява средно до **$n/2$** .



Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Пр2. Последователно търсене с преподреждане:

- Ако разполагаме с **предварителна информация** относно вероятността за достъп до всеки от елементите, бихме могли да ги подредим така, че **най-често търсеният да бъде в началото** на масива, следващия - непосредствено след него и т. н.

Използването на подобна стратегия е особено ефективно при силно неравномерно разпределение, при което **малък брой елементи се търсят с много голяма вероятност.**





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

- Ако не разполагаме с такава предварителна информация, бихме могли сами да си я получим с помощта на прости **статистически наблюдения**: достатъчно е **към всеки елемент да добавим брояч на достъп до него**. При всяко ново търсене ще актуализираме брояча на търсения елемент. Елементите са наредени по своите броячи, така след актуализацията той ще мине по-напред в списъка. Т. к. в списъка може да има многократно дублиране на честотите на достъп, в случай на размяна се налага съответно сравнение и със съседните елементи, докато се намери точната позиция, съответстваща на новата честота на елемента. Реорганизацията става за **време, пропорционално** на **n** , като в най-лошия случай може да се наложи последния елемент в масива да стане първи, разменяйки се последователно с всички преди него.





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Предложената схема с поддържане на броячи е доста **тромава** и изисква допълнителна памет от порядъка на **n** , но тя би могла да се оптимизира.

В този случай се оказва изключително ефективно прилагането на една проста стратегия за реорганизация, не изискваща поддържане на броячи и водене на статистика. Вместо това, при **всяко успешно търсене на елемент той се поставя в началото на масива**.

Разбира се, тази стратегия не ни гарантира оптимално подреждане на елементите съгласно честотата им на достъп, но пък е лесна за поддържане и достатъчно ефективна.





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Реорганизацията на масива става сравнително бързо, и колкото по-често е осъществяван достъп до даден елемент, с толкова по-голяма вероятност той ще се намира сред първите елементи на масива:

```
int search_reordering(int a[], int x )  
{  
    int rab, i=0;  
    while ((a[i]!=x)&&(i<num)) i++;  
    if (a[i]==x)  
        {rab=a[0]; a[0]=a[i]; a[i]=rab; return 1;}  
    else return 0;  
}
```



Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Пр3. *Търсене със стъпка. Квадратично търсене*

Ако сравним първите два алгоритъма за търсене (*последователно търсене и последователно търсене в подреден масив/таблица*), то може да се каже, че вторият алгоритъм не използва достатъчно пълноценно наредбата на елементите.

Ще поправим тази грешка, възприемайки следния подход:

Нека **изберем** някаква **стъпка k** и последователно извършим **проверка** дали търсеният **елемент е по-голям от първия елемент в масива**, от **$(k + 1)$ -ия елемент**, от **$(2k + 1)$ -ия елемент**, от **$(3k + 1)$ -ия елемент** и т. н. Процесът **приключва** при достигане на **елемент, по-голям или равен на търсения x** , или на края на масива.





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Да разгледаме по-внимателно долната схема (която се нарича *търсене със стъпка*). Нека предположим, че, прилагайки я, сме достигнали до *елемент, по-голям от x* . Сега можем да използваме последователно търсене в интервала, определен от последните две проби. В случай, че сме били излезли извън масива, можем да използваме последователно търсене от предишната проба до края на масива. Подобен подход води до силно съкращаване на броя на елементите, които се преглеждат от последователно търсене. Изложеният метод е обобщение на линейното търсене ($k=1$):

$a[1]$ $a[2]$ $a[3]$... $a[k+1]$... $a[2k+1]$... $a[n]$





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

В описания метод търсенето **започва от първия елемент** (или $a[1]$).

Доказано е, че този **избор**, както и $a[n]$, е **лош**, понеже носи минимална информация.

Много по-удачно е да се започне направо с $a[k]$. (Пак се вижда, че при $k=1$ се получава линейно търсене.)

В най-лошия случай търсения елемент е в последния интервал, което означава, че ще бъдат необходими $\lceil n/k \rceil$ сравнения, за да определим нужния ни интервал, в който да приложим линейното търсене. Към тях следва да се добави дължината на интервала, която при n , кратно на k , е $k-1$. Оттук следва, че в **най-лошия случай при търсене със стъпка k се извършват не повече от $\lceil n/k \rceil + k - 1$ сравнения.**





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Примерна реализация на описания алгоритъм:

```
unsigned seq_search(unsigned lf, unsigned rt, int key)
{
    while (lf<rt)
        if (a[lf++]==key)
            return lf-1;                //елементът е намерен
    return -1;                        // елементът не е намерен
}

unsigned jmp_search(int key, unsigned step)
{ unsigned ind;
  for (ind=0; ind<n&& a[ind]<key; ind+=step);
  return seq_search(ind+1<step ? 0 : ind+1- step, n<ind ? n: ind, key);
}
```



Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

Максимален брой сравнения при различни стойности на n и k :

n/k	1	2	3	4	5	6	7	8
1	1							
2	2	2						
3	3	2	3					
4	4	3	3	4				
5	5	3	3	4	5			
6	6	4	4	4	5	6		
7	7	4	4	4	5	6	7	
8	8	5	4	5	5	6	7	8

От таблицата се вижда, че **най-добрите стойности на k** са близки до **$n/2$** , т. е. са **разположени в средата** на съответния ред на таблицата или непосредствено вляво от нея.





Алгоритми за търсене. Сортиран масив.

Примери за последователно търсене:

За да определим по-точно, **кои стойности на k са най-добри** и **как зависят от n** , следва да определим при каква зависимост между k и n функцията **$f(k)$ приема минимална стойност:**

$$f(k) = \lceil n/k \rceil + k - 1$$

Може да се докаже (чрез изследване на знака на втората производна), че това е стойността **$k = \sqrt{n}$** . Тогава **$f(k) = 2\sqrt{n} + 1$** . В този случай **търсенето** се нарича **квадратично**.

Следователно сложността е **подобрена** - от n до $\sqrt{n}$



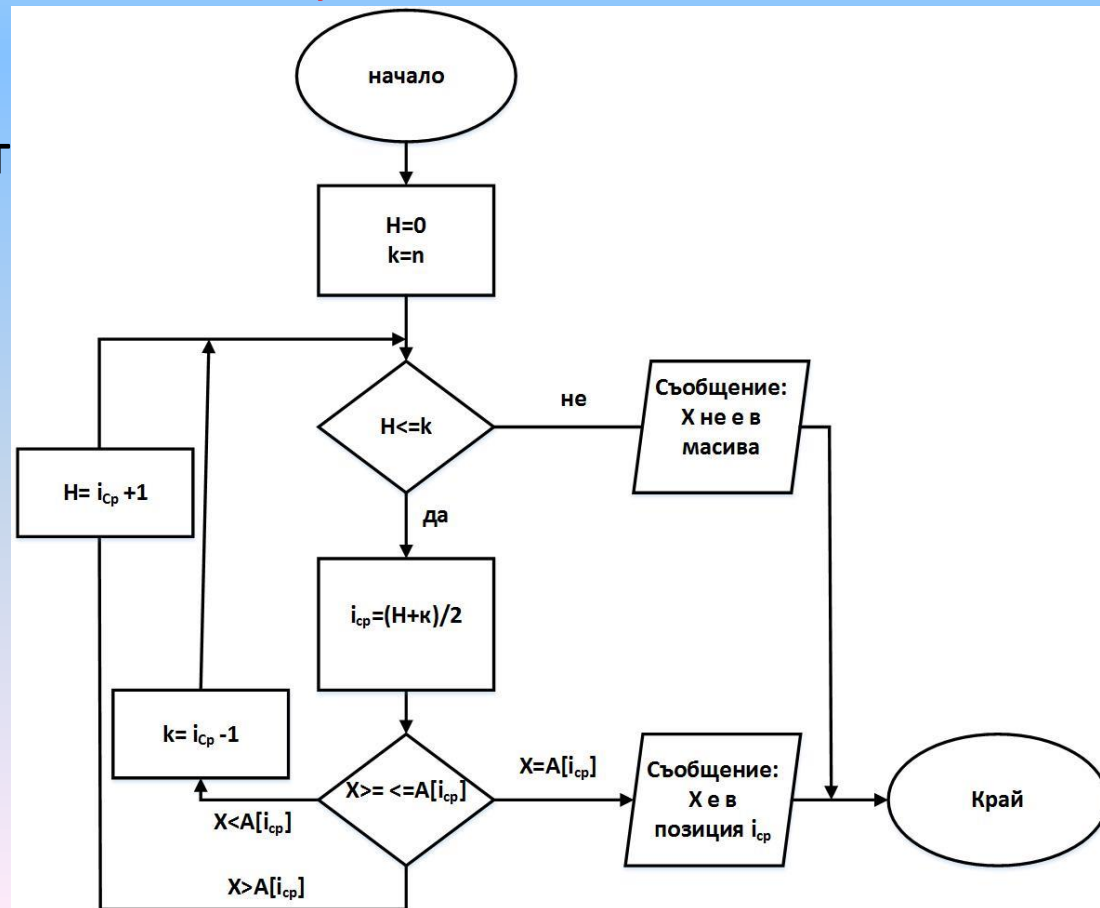
Алгоритми за търсене. Сортиран масив.

3. **Двоично търсене** – в участъка с **начало H** и **край k** (в началото $H=0$ и $k=n$) **се избира среден елемент $A[i_{cp}]$** , където $i_{cp}=(H+k)/2$.

Търсения елемент **x** **се сравнява с $A[i_{cp}]$** и **ако $x=A[i_{cp}]$** , тогава търсенят елемент е намерен и следва **край**.

Ако **$x < A[i_{cp}]$** , тогава търсене-
то продължава в лявата част
с двойно по-малък размер,
началото **H** **се запазва**, а
 $k=i_{cp}-1$.

Ако **$x > A[i_{cp}]$** , тогава търсене-
то продължава в дясна част
с двойно по-малък размер,
началото е **$H=i_{cp}+1$** , а
 k **се запазва**.





Алгоритми за търсене. Сортиран масив.

1 2 3 4 5 6 7 8 9

Tutorial4us.com

На всяка стъпка размерът на полето за търсене се съкращава приблизително двойно:

```
int Bin_search(int a[], int lf, int rt, int k)
//a[] - подредена последователност от num цели числа
//k - търсената стойност
//lf - left - лявата граница на индекса
//rt - right - дясната граница на индекса
//Обръщението към функцията изглежда така:
```

```
Bin_search(a, 0, num, k)
{ if (lf>rt) return 0; int m=(lf+rt)/2;
  if (a[m]==k) return a[m];
  if (lf==rt) return 0;
  if(a[m]>k) return Bin_search(a, lf, m-1, k);
    else return Bin_search(a, m+1, rt, k);
```

}



Алгоритми за търсене. Сортиран масив.

Ситуацията **lf=rt** отговаря на листата на дървото. Алгоритъмът предполага максимум **$[\log_2 n] + 1$** сравнения.

Сложността на алгоритъма е **$O(\log n)$** .

Недостатък на двоичното търсене е тежката **операция деление**, извършвана на всяка стъпка от изпълнението на алгоритъма.

Реализацията му на C/C++ позволява да се избегне тази операция, като делението на 2 се извършва с изместване на делимото с една позиция надясно, т. е. Редът

```
int m=(lf+rt)/2;
```

следва **да се замени** с:

```
int m=(lf+rt)>>1;
```





Алгоритми за търсене. Сортиран масив.

Описаният процес е рекурсивен. На практика обаче на всяка стъпка се разглежда точно един от двата подмасива. Вземайки предвид това, не е трудно да реализираме **итеративно решение**:

```
int bin_search (int k)
{
    int lf=0, rt=n-1, m;
    while (lf <=rt)
    {
        m = (lf + rt)>>1;
        if (k <a[m])
            rt = m - 1;
        else if (k>a[m])
            lf = m + 1;
        else return m;        // m - индекс на намерения елемент
    }
    return (-1);              // елементът не е намерен
}
```




Алгоритми за търсене. Сортиран масив.

Пр.1: Фибоначиево търсене - модификация на двоичното търсене. За разлика от двоичното търсене, тук **за сравнение се избира** не средния елемент в масива, а **елемент с индекс F_{k-1}** , като се има предвид, че **броят на елементите в масива $n = F_k - 1$** .

Числата на Фибоначи се задават с рекурентната **формула**:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, n \geq 2$$

На **първа стъпка** сравняваме търсената стойност **key** с **$a[F_{k-1}]$** . В случай на равенство **търсенето приключва** успешно.

Ако **$key < a[F_{k-1}]$** , на следваща стъпка се разглеждат елементите с индексите **0, 1, 2, ..., $F_{k-1} - 1$** . Ако пък **$key > a[F_{k-1}]$** , разглежданите елементи ще имат индекси **$F_{k-1} + 1, F_{k-1} + 2, ..., n$** . В 1-я случай за следваща стъпка получаваме последователност с дължина **$F_{k-1} - 1$** , а във втория - **$F_{k-2} - 1$** . Процесът се повтаря върху неотхвърлената част до намиране на търсения елемент или до достигане на празната последователност.





Алгоритми за търсене. Сортиран масив.

Една възможна реализация на Фибоначиево търсене изглежда така:

- Функция за търсене на числата на Фибоначи:

```
int fib[n];          //помощен масив, в който се съхраняват
                     //числата на Фибоначи
int findFib(int n) //функция за търсене на числата
{  int k;  fib[0]=0; fib[1]=1;
   for (k=2; ;k++)
       if ((fib[k]=fib[k-1]+fib[k-2]) > n)
           return k-1;
   return 0;
}
```





Алгоритми за търсене. Сортиран масив.

- Функция за Фибоначиево търсене:

```
int fibsearch(int key, int n)  //Фибоначиево търсене
{ int p, q, r, k;
  k=findFib(n);
  p=fib[k-1]; q=fib[k-2]; r=fib[k-3];
  if (key>a[p]) p+=n-fib[k]+1;
  while (p>0)
    if (key==a[p]) return p;
    else
      if (key<a[p])
        if (0==r) p=0;
        else
          { int t; p-=r; t=q; q=r; r=t-r; }
      else
        if (1==q) p=0;
        else
          { p+=r; q-=r; r-=q; }
    return 0;
```





Алгоритми за търсене. Сортиран масив.

Алгоритъмът не съдържа операцията деление, но ефективността му не е по-голяма от тази на двоичното търсене.

На практика алгоритъмът на Фибоначиево търсене изгражда балансирано дърво, което в най-лошия случай с **45%** е по-високо от дървото на двоичното търсене.

Алгоритми за търсене. Сортиран масив.

Пр2. **Интерполационно търсене** - също модификация на двоичното търсене, при което масивът се дели на две равни части и индексът на сравняваният елемент се изчислява като

$$m = (lf + rt) / 2 \quad (lf - \text{левия индекс}, rt - \text{десния индекс})$$

или $m = lf + (rt - lf) / 2$

Получената формула показва, че следващата позиция на разделяне на интервала се получава, като към началото му добавим половината от дължината му.

Знаейки търсената стойност **key**, ние можем да потърсим позицията ѝ по следната интерполационна формула:

$$m = lf + k * (rt - lf), \quad \text{където } k = (key - a[lf]) / (a[rt] - a[lf])$$

Коефициентът **k** **замества** константата **1/2** (в двоичното търсене) и ни дава приблизителната позиция на търсения елемент в разглеждания интервал.





Алгоритми за търсене. Сортиран масив.

- Функция за интерполационно търсене:

```
int inter_search(int key, int num)
{ unsigned lf, rt, m;                                //lf - началния индекс в интервала
                                                    //rt - крайния индекс в интервала
                                                    //m - индекса на сравнявания елемент
                                                    //коефициент lf=0; rt=num-1;

float k;
while (lf<=rt)
{
    if (a[rt]==a[lf])
        if (a[lf]==key)
            return lf;                                //елементът е намерен
        else
            return 0;                                //елементът не е намерен
    k=(float) (key-a[lf])/(a[rt]-a[lf]);              //коефициент на
позцията
    if (k<0||k>1)
        return 0;                                //елементът не е намерен
    m=(unsigned) (lf+k*(rt-lf)+0.5);
    if (key<a[m])
        rt=m-1;
    else if(key>a[m])
        lf=m+1;
    else return m;                                //елементът е намерен
}
return 0;}
```





Алгоритми за търсене. Сортиран масив.

Методът е реализуем, когато ключовите стойности са числа или могат лесно да се интерпретират като такива, така че коефициента k да попада в интервала $[0, 1]$.

Алгоритъмът е добър при равномерно разпределение на стойностите в масива. Тогава интерполационното търсене извършва по-малко от $\log_2(\log_2 n) + 1$ сравнения както при успешно, така и при неуспешно търсене. Препоръчва се при големи обеми данни и големи по размер ключове, когато сравненията отнемат много време.



Хеширане. Обща информация

Какво е хеш?

Hash на английски означава "каша, бъркотия", а като глагол - "нарязвам". Това в основен смисъл е начинът, по който се получават "псевдонимите" на входните обекти - те се нарязват и разбъркват по някакъв начин, и (малка част от) кашата, която се получи е така нареченият хеш. Важно е да се отбележи, че от "кашата" не може да се възстанови първоначалният обект - само се знае, че е получена от него.

Хеш стойност, е "псевдоним" на някакъв сложен обект. По псевдонима обикновено не можем да възстановим целия обект, но все пак можем да извършваме някои по-прости операции - като например да видим дали два обекта (например стринга) са еднакви или не. Най-често хешът е число, но в някои случаи може да бъдат две или повече числа, или последователност от битове.



Хеширане . Обща информация

Общо **хешът** е представяне на данни с произволен размер (един стринг може да има произволна дължина) чрез данни с фиксиран размер (размерът на `int` в байтове е фиксиран).

Основните изисквания към хеша е да бъде:

- **Лесно сравним** - обикновено хешовете се ползват за да се търсят дубликати (повтарящи се елементи) между обектите, които те представят
- **Компактен** - когато хешът е малък по размер, той може да бъде местен с минимален разход на време. Допълнително, тъй като обикновено имаме голям брой обекти, на които даваме хеш стойност, а така и съответно голям брой хешове, ако те са малки по размер няма да заемат голямо количество памет



Хеширане. Обща информация

Какво е хеширане?

Хеширане (от английски **hashing**) е действие, при което определен обект се асоциира с хеш стойност. Тъй като самото хеширане много зависи от обектите, върху които се прилага, а също така и какви хеш стойности ще се използват (може да са един int, няколко int-а, последователност от битове, или нещо друго), това е по-скоро **техника**, отколкото алгоритъм.

RANDY





Хеширане. Обща информация.

Защо ни трябва да хешираме?

Нека имаме стринга "Elly". На него можем да съпоставим числото (хеша) 1164733561. Допълнително **имаме стринговете "Stancho"** с хеш 1852008559 и **"Ellen"** с хеш 1819043182. Нека имаме масив с голям брой от тези стрингове и искаме да видим кой от тях се среща най-често. Ако ползваме самите стрингове, ще имаме много сравнения между "Elly" и "Elly", "Elly" и "Stancho", "Elly" и "Ellen", "Stancho" и "Stancho", "Stancho" и "Ellen", и "Ellen" и "Ellen". Докато при двойките "Elly" и "Stancho" и "Stancho" и "Ellen" бихме намерили разлика още на първата буква, при всички останали ще ни трябват по няколко сравнения докато намерим разлика или установим, че са един и същ стринг. **Например** при "Elly" и "Ellen" ще намерим разлика **чак на четвъртата буква**, а при "Stancho" и "Stancho" ще сравним всичките осем, за да установим, че е **един и същ стринг**.



Хеширане. Обща информация.

Защо ни трябва да хешираме?

От друга страна, ако сравнявахме техните хешове, щяхме да се нуждаем от **точно едно сравнение** за да видим дали стринговете са различни или не.

Ако имаме същата задача, но със стринговете "aaaaaaaaaaaaaaaaaaaaa", "aaaaaaaaaaaaaaaaaaaab", и "aaaaaaaaaaaaaaaaaaaac", разликата ще е още по-осезаема - **решението с хешове ще е двадесет пъти по-бързо!**

Може да се види, че ако максималната дължина на стринговете е $O(N)$, решението с хешове ще е около **N пъти по-бързо.**



Хеширане. Обща информация.

Хешираща функция

Хеширането - как точно го правим? Как решихме, че хешът на "Elly" е 1164733561, а този на "Stancho" е 1852008559?

Реално, в зависимост от входните обекти (какво искаме да хешираме) се ползват различни **хеширащи функции**. Хешираща функция е функция, която приема като аргумент обект и връща неговия хеш. Логиката в тях е различна именно в зависимост какви са обектите. **Например** ползва се различна логика ако се хешират стрингове и ако се хешират дати.

Един прост начин да се хешират стрингове е да се разглеждат като числа в 256-тична бройна система (ако се счита, че стринговете се състоят само от ASCII символи, може да се ползват техните ASCII кодове като "цифри" в тази система).



Алгоритми за търсене. Хеширане

Разгледаните методи за търсене предполагат, че търсеният елемент се съхранява в таблица и за откриването му е необходимо да се организира преглеждане на определена част от таблицата или в най-лошия случай - на цялата таблица.

Хеширането е метод за организация на таблицата, осигуряващ бързо намиране на търсения елемент и таблично преобразуване.

Таблицата в която се разполагат елементите се нарича **хеш-таблица**



Алгоритми за търсене. Хеширане

Разполагането на елементи в хеш - таблицата се осъществява чрез **специално преобразуване на техните ключове**.

Всеки елемент на хеш-таблицата се характеризира с **две полета** - ключ **key** и данни **data**.

Ключът на елемента може да бъде произволна структура, идентифицираща го уникално.

Функцията, която **трансформира ключ в табличен индекс**, се нарича **хеш-функция**. Тя трябва да отговаря на две основни **условия**:

- да не отнема много изчислително време;
- да дава стойности, които относително равномерно покриват цялото множество индекси в хеш-таблицата.



Алгоритми за търсене. Хеширане

Когато **ключът е целочислен**, най-често хеш-функциите **$H(K)$** се задават като **остатък от деление на ключа K върху M** , което определя размера на таблицата:

$$H(K) = K \bmod M \quad 0 \leq H(K_i) \leq M-1$$

В случая, когато **ключовете са символни низове**, те се преобразуват в цели положителни числа чрез сумиране на **ASCII-кодовете** на влизащите в тях символи.

R	A	N	D	Y
82	65	78	68	89

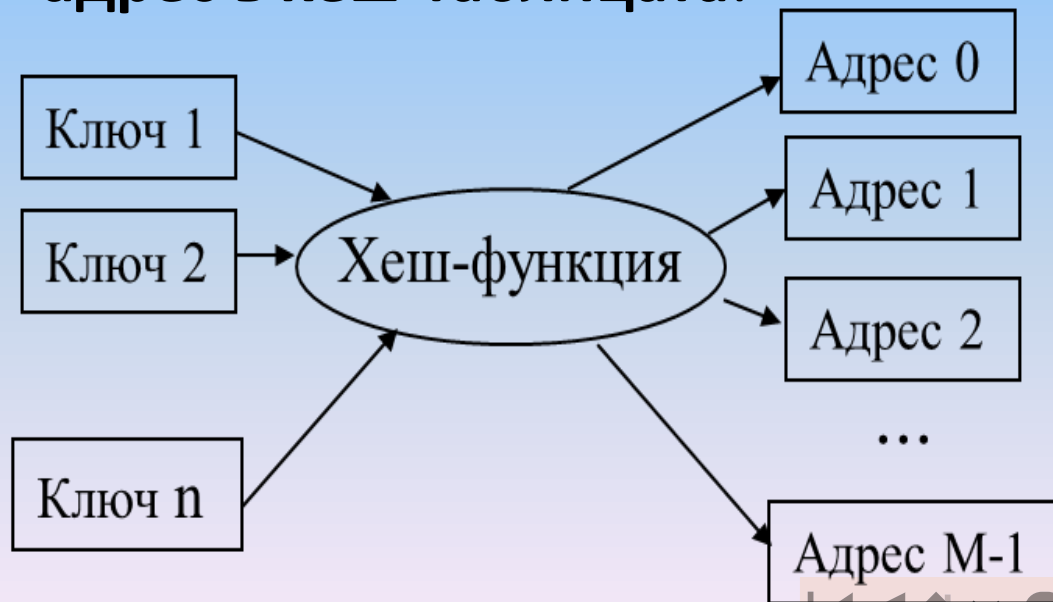


Алгоритми за търсене. Хеширане

Хеш-таблица:

0	...	...
	$H(K_i)$	$A[i]$
	...	...
	...	...
	$H(K_j)$	$A[j]$
	...	...
	$H(K_m)$	$A[m]$
	...	...
	...	...
M-1		

Може да се каже, че **хеширането е процес**, при който по зададен ключ на елемента се определя неговия адрес в хеш-таблицата:





Алгоритми за търсене. Хеширане

Методът е **много бърз**, т. к. осигурява **директен достъп** до търсения елемент чрез неговата хеш-функция.

При използването на този подход се очаква **константно време** за операциите **търсене** и **вмъкване** в таблицата, независимо какъв е нейния размер.

Някои автори дефинират хеш-таблицата като структура от данни, за която е характерен директен достъп до елементите, независимо от типа им.

E R I C

$$(69 + 82 + 73 + 67) \% 100 = 91$$



Insert to hash table



Алгоритми за търсене. Хеширане

Например, таблица, съдържаща информация за имена и факултетни номера на студенти от една студентска група. Факултетните номера уникално определят всеки елемент в таблицата, затова се използват като ключове:

$026760 \% 30 \rightarrow$	0	026760	Иван Иванов Иванов
...		...	
$026775 \% 30 \rightarrow$	15	026775	Вяра Янкова Павлова
...		...	
$026728 \% 30 \rightarrow$	28	026728	Ана Илиева Илиева
$026819 \% 30 \rightarrow$	29	026819	Петър Тодоров Пеев

$$026733 \% 30 = 3$$

$$026793 \% 30 = 3$$



Алгоритми за търсене. Хеширане

Недостатък на хеширането: Нека да предположим, че съществуват два елемента $A[i]$ и $A[j]$ с ключове K_i и K_j , които имат една и съща стойност на хеш-функцията: $H(K_i) = H(K_j)$.

Това означава, че двата елемента би трябвало да се запишат с един и същи индекс в хеш-таблицата, което не е възможно.

Подобна ситуация се нарича **колизия**.

Следователно е важен начина, по който се избира хеш-функция.

Хеш-функцията трябва да **минимизира броя** на възможните **колизии** и да разпределя елементите равномерно в таблицата. Ако таблицата е съществено по-голяма от реалния брой елементи, това намалява вероятността от възникване на колизии, но използването ѝ ще бъде неефективно от гледна точка на заемана памет.



Алгоритми за търсене. Хеширане

Класически хеш-функции

Нека ключовете са цели числа. Ако не са, ще приемем вече споменатото правило: за ключове-символни низове ще сумираме ASCII-кодовете на влизащите в тях символи.

Нека е дадена хеш-таблица с **капацитет n** и **елемент с ключ k** . Най-често срещаните в практиката хеш-функции (върху целочислен ключ) са следните:

1. Остатък при деление с размера на таблицата

Това е прост и доста ефективен начин за хеширане: **ключът k се разделя целочислено на n и се взема остатъкът от делението**. При този подход **не е удачно n да бъде степен на 2**, т. к. в този случай хеш-кодът се определя от **младшите битове на k** . Добре е хеш-кода да зависи от всички битове на ключа. За тази цел **за n най-често се избира просто число**.



Алгоритми за търсене. Хеширане

Класически хеш-функции

2. Мултипликативно хеширане

Това е друг широко разпространен начин. Избира се реална константа b : $0 < b < 1$

За даден ключ k хеш-функцията има вид: $H(k) = [n * \{k * b\}]$

където $\{k * b\}$ означава дробната част на реалното число, т. е.

$$k * b - [k * b].$$

Въпреки, че изборът на константата b (при ограничението $0 < b < 1$) е произволен, за някои стойности практическите резултати са добри. Д. Кнут (D. Knuth. *The Art of Computer Programming*, 1998) предлага за b да се използва златното сечение:

$$b = \frac{\sqrt{5} - 1}{2} = 0,6180339887...$$



Алгоритми за търсене. Хеширане

Класически хеш-функции

3. Хеш-функции върху части от ключа

а) **Извличане на цифри** - При тази схема **от ключа се извличат само цифри**, намиращи се на определени позиции (например - първата, третата и петата цифра от числото). Така за ключовете 123659, 425435, 546754, 676576 ще получим хеш-адреси съответно 135, 453, 565 и 667. В този случай **н трябва да е 1000**. Методът е добър, ако числата не съдържат повтарящи се цифри.

б) **Сгъване** - Този метод се прилага най-често, когато ключовете са много големи числа. Възможни са разновидности, но като цяло всички се **основават на разделяне на ключа на части** и извършване на някакви аритметични действия върху получените части. Например, числото може да се раздели на две (или три и повече) части и сумата от получените числа да определя хеш-адреса.



Алгоритми за търсене. Хеширане

Класически хеш-функции

в) Повдигане на средата в квадрат - Тази схема се основава на извличане на средните p цифри на ключа и повдигането им в квадрат.

Например, за ключа 125657134280980 средните три цифри са 134, квадратът на числото е 17956.

Ако резултатът надхвърля n , се премахват първите няколко значещи цифри.

Например, ако $n = 10001$, то от 17956 се премахва първата цифра и полученият хеш-адрес е 7956.

(Последната операция не е еквивалента на намиране на остатъка при деление на n .)



Алгоритми за търсене. Хеширане

Класически хеш-функции

4. Хеш-функции върху символни низове

Символните низове са най-често хешираният тип данни.

Намирането на добра хеш-функция за тях е сериозен проблем.

Ще използваме следната схема:

```
result = инициализиране();  
while (c = следващ_символ)  
{  
    result=комбиниране (result, c);  
    result=вътрешно_модифициране(result);  
}  
result=допълнително_модифициране (result);
```



Алгоритми за търсене. Хеширане

Адитивно хеширане е най-разпространено (класическо), но най-неефективното хеширане. Сумират се ASCII-кодовете на символите и сумата взета по модул определя размера на масива:

```
unsigned long hash_function(const char *key, unsigned long size)  
{ unsigned long result = 0;  
  while (*key)  
    result+=(unsigned char) *key++;  
  return result % size; }
```

Обикновено в дължината на низа се включва сумата, за да може изрично да влияе на хеш-кода:

```
unsigned long hash_function(const char *key, unsigned long size)  
{ unsigned long result = strlen(key);  
  while (*key)  
    result+=(unsigned char) *key++;  
  return result % size; }
```



Алгоритми за търсене. Хеширане

Дължината на символния низ би могла да се подава като параметър, което спестява обръщението към функцията ***strlen(key)***. Или просто да се получи като разлика между текущата позиция преди и след обхождането на низа:

```
unsigned long hash_function(const char *key, unsigned long size)  
{ const char * save_key=key;  
  unsigned long result = 0;  
  while (*key)  
    result+=(unsigned char) *key++;  
  result +=save_key - key;  
  return result % size; }
```

Размерът на масива най-често се избира да бъде просто число. Понякога обаче се използва масив с размер степен на 2, при което последния ред може да се опрости до:

```
return result & (size - 1);
```



Алгоритми за търсене. Хеширане

Други подходи при работа със символни низове:

- **ротирано хеширане** - вместо сумиране се използват само поразредени операции. Размерът на масива трябва да бъде просто число;
- **хеширане едно по едно** - вариация на горното, но с допълнителни манипулации върху *result*;
- **хеширане по Пиърсън** - използва допълнителен масив, съдържащ пермутации на числата от 0 до 255. Полученият хеш-код е еднобайтов: от 0 до 255. По-голям код може да се получи, ако функцията се извиква няколко пъти с различни масиви, при което всяко извикване ще дава един байт от резултата;
- **хеширане по CRC (Cyclic Redundancy Check — проверка на цикличния остатък, контролна сума)** - стойностите в масива се генерират като псевдослучайна двоична последователност, удовлетворяваща определени условия;
- **други** класически хеш-функции



Алгоритми за търсене. Хеширане

Каквато и хеш-функция да изберем, в най-лошия случай тя ще **предизвика n колизии**, където n е размерът на хеш-таблицата.

В **най-добрия случай** ще възникнат **0 колизии**. За да се предпазим от колизии, трябва да гарантираме **равномерно разпределение на множеството от допустимите стойности на ключовете върху множеството от индексите на масива/таблицата**. Тогава вероятността за съпоставяне на един и същи индекс в масива за два различни ключа, т. е. за възникване на колизия, ще бъде **$1/n$** .

Една от възможностите е да се комбинират няколко хеш-функции, които разпределят елементите равномерно.



И все пак, как да се справим с колизиите, когато те възникват?



Алгоритми за търсене. Хеширане

Справяне с колизии

Нека е дадена хеш-таблица с размер $n=10$.

Ще използваме проста хеш-функция, която се основава на извличане на цифри - **първата цифра на ключа** ще дава **хеш-адреса** на елемента.

Например, искаме да включим в таблицата елементите 234, 235, 567, 123, 534 и 647.

Всички методи условно могат да бъдат разделени на **две групи**:

- **затворено хеширане**
- **отворено хеширане**



Алгоритми за търсене. Хеширане

1. Затворено хеширане

а) Линејно пробване - при този подход разполагаме с **едно основно място, където се съхраняват данните (n слота)**.

Когато настъпи колизия, пробваме последователно да променяме получения хеш-адрес, докато стигнем до свободен слот (вход в таблицата). Промяната се прави по предварително дефинирана схема.

Например, възможно е да **увеличаваме адреса** с някакво естествено число **s ($0 < s < n$)**. Ако **адресът стане равен на n** , продължаваме търсенето в началото на хеш-таблицата, като вземаме **остатъка по модул n** . За да може при подобно увеличаване да се обходят всички адреси на хеш-таблицата, трябва **n и s да бъдат взаимно прости, т. е. $\text{НГОД}(n, s) = 1$** .



Алгоритми за търсене. Хеширане

Очевидно, при този подход не могат да се включват повече елементи, от колкото е капацитетът на хеш-таблицата.

Когато тя се препълни, е необходимо разширяване (заделяне на памет за по-голям масив и освобождаване на заетата от текущия масив памет).

На практика запълването на таблицата при този вид хеширане **не трябва да превишава $1/2$** .



Алгоритми за търсене. Хеширане

Последователно включване на елементи с разрешаване на колизиите чрез линейно пробване със стъпка 1:

		234	235	567	534	123	647
0							
1						123	123
2		234	234↓	234	234	234	234
3			235↓	235	235	235	235
4							
5				567	567↓	567	567
6					534↓	534	534↓
7							647↓
8							
9							



Алгоритми за търсене. Хеширане

При търсенето на елемент по ключ се процедира по аналогичен начин - **първо се изчислява хеш-адресът** с помощта на хеш-функцията и в случай, че **не се попадне на търсения ключ**, се прави линейно пробване.

Линейното пробване се състои в **променяне на адреса с 1**, докато се стигне до търсеният елемент (като се предполага, че ключът е в таблицата).

В този случай можем да се попадне на ключове, при които хеширането да се окаже неефективно. *Например, когато елементите образуват групи с близки хеш-кодове (т. н. клъстери).* Тогава в процеса на разрешаване на колизия възникват серия нови (**вторични**) колизии.



Алгоритми за търсене. Хеширане

б) Двойно хеширане

Двойното хеширане е подобно на вече разгледаните методи, но използва две хеш-функции:

$$h(k, i) = h_1(k) + i * h_2(k)$$

Тук i е номерът на пробата след първата колизия. Втората хеш-функция се използва само в случаите, когато резултатът от първата води до колизия. Като ключ при повторно хеширане може да се използва както оригиналният ключ на елемента, така и хеш-адресът, получен при прилагане на първата хеш-функция.



Алгоритми за търсене. Хеширане

в) Рехеширане

Прилага се **при голямо запълване на таблицата (над $1/2$)**. Създава се нова хеш-таблица с двойно по-голям размер (обикновено се взема първото просто число, по-голямо от $2n$). В новата таблица се прехвърлят ключовете от първата.

г) Разширено хеширане

Използва се тогава, когато **хеш-таблицата не може да се помести в оперативната памет**. В този случай таблицата се **зарежда в ОП на части (по блокове)**. Хеширането трябва да е такова, че да има минимален среден брой обръщения към външна памет при търсене на ключ в таблицата.



Алгоритми за търсене. Хеширане

2. Отворено хеширане

а) Допълнителна памет за колизии

При тази схема се отделя допълнителна памет за обработване на колизии. Всеки елемент, попаднал в колизия, се разполага на първото свободно място в допълнително отделената памет:





Алгоритми за търсене. Хеширане

При търсене в хеш-таблицата първо се проверява адресът, получен чрез хеш-функцията. Ако елементът с търсения ключ не е там, се претърсва цялата допълнителна памет.

		234	235	567	534	123	647
0							
1						123	123
2		234	234	234	234	234	234
3							
4							
5				567	567	567	567
6							647
7							
8							
9							
10			235	235	235	235	235
11					534	534	534
...	...	...	...	...	...	...	...

✗



Алгоритми за търсене. Хеширане

б) Списък на препълванията

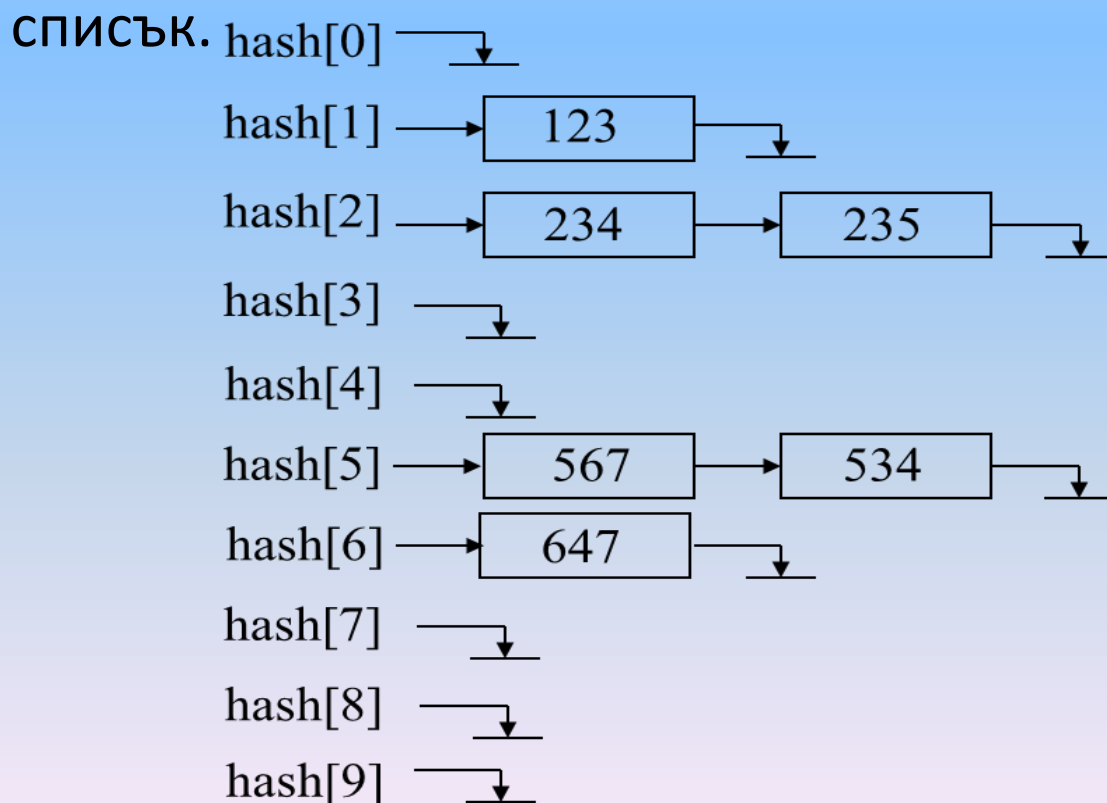
Възможно е да се използва и **динамично заделяна памет** (*heap*). Елементите със съвпадащи хеш-адреси се разполагат в списъци на препълванията. **Всеки слот** на хеш-таблицата **е указател към динамичен списък**, съдържащ само членове на съответния клас синоними. Самата хеш-таблица е **масив от указатели**. При включване на нов елемент той се добавя в началото на списъка, намиращ се на слота, определен от хеш-функцията. Проблемът с колизии отпада.





Алгоритми за търсене. Хеширане

При търсене се преглежда списъкът, определен от хеш-функцията. Сложността на операцията търсене зависи от броя на елементите, с които дадения е в колизия, т. е. от дължината на съответния



Елементите могат да се пазят и в друга динамична структура, например - в BST.



Алгоритми за търсене. Хеширане

Пример за създаване хеш-таблица и търсене в нея.

Данните в таблицата са имена и факултетни номера на студенти от една студентска група. Ключовите стойности на елементите са факултетните номера. Възможните колизии се отстраняват чрез използване на списъците на препълванията.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
const m=31; //размер на хеш-таблицата
```

```
struct ptr //(студентската група)
```

```
{char name[10]; int fn; ptr *next;} *hash[m]; //структура на данните
```

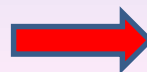
```
//hash[m] - хеш-таблица, реализирана като масив от указатели,
```

```
//сочещи към списъците на препълванията
```

```
void init() //инициализация на таблицата
```

```
{ for (int i=0; i<m; i++)
```

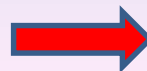
```
hash[i]=NULL; }
```





Алгоритми за търсене. Хеширане

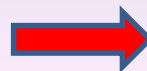
```
void create()                                //създаване на хеш-таблицата
{
    int f, h;                                // f - текущ Ф.№, h - текуща стойност на хеш-функцията
    cout<<"\nЗадайте Ф.№: ";
    cin>>f;
    h=f%m;
    ptr *q=hash[h];                          //работен указател
    ptr *s=new ptr;                          //работен указател
    s->fn=f;
    cout<<"\nВъведете име: ";
    cin>>s->name;
    s->next=NULL;
    if (!q)                                  //няма колизия, списъкът е празен
        hash[h]=s;
    else                                     //списъкът не е празен
    {
        while (q->next) q=q->next;
        q->next=s;
    }
}
```





Алгоритми за търсене. Хеширане

```
void seek()                                //търсене в хеш-таблицата
{
    int fl=1; //флаг
    int f, h;
    cout<<"\nЗадайте търсения Ф.№:";
    cin>>f;
    h=f % m;                               //изчисляване на хеш-функцията
    ptr *q=hash[h];
    while (q&&fl)
        if (q->fn==f)
            {fl=0; cout<<q->name<<" " <<q->fn;}
        else q=q->next;
    if (fl)
        cout<<"\nВ групата няма студент с Ф.№ " <<f<<"!";
}
```





Алгоритми за търсене. Хеширане

```
void main()  
{init();           //инициализация  
  char c;           //работна променлива  
  do  
  {  
    create();        //добавяне на елемент в таблицата  
    cout<<"Ще добавяте ли друг студент (Y/N):";  
    cin>>c; }  
  while (c!='N'&&c!='n');  
  do  
  {  seek();  
    cout<<"\nИскате ли да потърсите друг студент (Y/N):";  
    cin>>c; }  
  while (c!='N'&&c!='n');  
}
```



Алгоритми за търсене. Хеширане

Заклучение:

Съотношението между броя ключовете, записани в хеш-таблицата, и нейния размер се нарича *фактор на запълване* (или на *натоварване*).

При *отвореното* хеширане *факторът трябва* да има *стойност, близка до 1* (може и по-голяма от 1).

При *затвореното* хеширане *факторът на запълване не трябва да надвишава 1/2*.



Алгоритми за търсене. Хеширане

Къде се прилага хеширането?

Навсякъде, където се извършва търсене и запис на данни, когато е **от значение времето**, необходимо за реализация на тези две операции.

Например:

- ***при компилаторите*** (в т.н. символна таблица, за да се съхраняват в нея идентификатори в изходния код на компилираната програма;
- в ***програмите за игри*** - в т.н. транспозиционни таблици;
- в ***програмите за проверка на правописа*** - при търсене на думите в речник



Алгоритми за търсене. Хеширане

- **Пазене на пароли** - голяма част от сайтовете, които изискват парола, **не пазят самата парола** на сървърите си, а само **нейн хеш**. Всеки път, когато се log-вате на сайта, въведената от вас парола се хешира и се изпраща на сайта, а там хешът се сравнява със запазения при регистрацията хеш. Така дори някой да се добере до вашата парола на сайта, той няма да има лесен начин да разбере *каква е самата парола* и да пробва да влезе с вашето потребителско име и паролата в друг сайт.

- **Верификация на файлове** - Когато теглите изпълним файл (например .exe) от даден сайт, фирмата производител предоставя за него хеш, с който можете да проверите, че файлът не е променян (примерно не е заразен с вирус). Така теглите файла, намирате неговия очакван хеш от производителя, хеширате изтегления файл и сравнявате хешовете. Ако те са еднакви, значи файлът не е променян. Ако не са еднакви, значи или е имало проблем при тегленето, или файлът е инфектиран с нещо.



Алгоритми за търсене. Хеширане

- **Internet Cache** - когато отворите някой сайт, има голям шанс част от него да не бъде променен от последното му зареждане. Най-често това са картинки и бутони, flash картинки/игрички и други. Тези файлове, които са малко вероятно да бъдат променени, биват кеширани на друго ниво - например на някой сървър по пътя от вашия компютър до сайта, или дори на вашия компютър. Когато отворите страница за първи път, цялото нейно статично (непроменливо) съдържание се сваля на компютъра ви, включително картинките, ако има такива. Следващият път, когато отворите сайта (и кешът на компютъра ви не се е изтрил), картинките, например, няма да бъдат свалени наново от сървъра (и да обиколят половината свят за да стигнат до вас), а ще бъдат показани вече свалената такава! Реално компютърът ви знае, че ги има свалени, като пази хеш на техния адрес.



Алгоритми за търсене. Хеширане

- **Bloom Filters** - друга честа употреба на хеширането са така наречените *Bloom Filters*. Те се ползват за бърза проверка дали даден обект **не принадлежи** на дадено множество. Например ако имаме огромна база данни (или огромен файл с някаква информация) и търсим в него даден обект, това би могло да бъде много бавно - особено ако обектът го няма и се наложи да обходим всичките данни. Блуум филтърът представлява комбинация на хешовете на всички обекти в базата данни (или файла), която ни позволява (хеширайки обекта, който търсим) да проверим дали него *със сигурност го няма* там - в който случай не би се наложило изобщо да го търсим. Ако пък блуум филтърът каже, че е възможно да го има, ще трябва да направим отново стандартното (бавно) търсене. Забележете, че е възможно отново да се обходят всички данни и да не го намерим - това би се случило ако е имало колизия в блуум филтъра.